

Malware Similarity Trends and Gaps Analysis

Final Presentation



Team:

Anmol Agarwal

Gina Fossati

Jennifer Ward

Mentors:

Dr. Bhavani Thuraisingham

Brian Ricks

MITRE

Sponsor:

Frank Posluszny

Principal Cyber Security Engineer

The MITRE Corporation

Outline

- Overview
- Completed Work
- Results
- Analysis
- Challenges
- Future Work
- References

Overview

- The main goal of this project is to develop a prototype of an **automated tool** that analyzes malicious **executables** and outputs useful information about the malware.
- To achieve this, we researched the **malware development life cycle** and each wrote two implementations of an **exfiltration tool** in various languages based on provided requirements to provide a ground truth for our feature extraction tool.
- We compared the final product of our malware tools, in the form of **Windows executables**, to find similarities and differences evident from **each phase of the MDLC**.
- With this knowledge, we were able to develop our prototype to **extract these features automatically** from a piece of malware.

Work Completed: Exfiltration Tools

- **Exfiltration Tools:** The team has developed variations of an exfiltration tool based on provided requirements that accomplishes the following tasks:
 - Extracts files specified by the attacker from a victim's machine.
 - Obfuscates these files.
 - Sends the obfuscated files to an IP address or FQDN through a HTTP Post command.
- **Deobfuscation Tools:** The tool comes with an additional deobfuscation tool or option so that once the adversary retrieves these files from the IP address or FQDN, the exfiltrated files are deobfuscated.
- **Variations:** The team has developed six exfiltration tools, in which each team member developed two implementations in two different languages.
 - Three variations in Python.
 - Two variations in Java.
 - One variation in Go.

Work Completed: Packaging

Language	Go			
Tool	go build package			
Options	Standard	Switching off Optimization and inline	Disable DWARF debugging	Switching off Debug Symbols
Command	go build tool.go	go build -gcflags="-N -l" tool.go	go build -ldflags="-w" tool.go	go build -ldflags="-s" tool.go

Language	Python 3			
Tool	cx_freeze	PyInstaller (Python 2.7)**		
Options	*setup.py	Standard	Compiler Optimization	Encrypted
Command	python3 setup.py build	python -m PyInstaller --onefile exfilTool.py	python -O -m PyInstaller --onefile exfilTool.py	python -m PyInstaller --key=1234 --onefile exfilTool.py

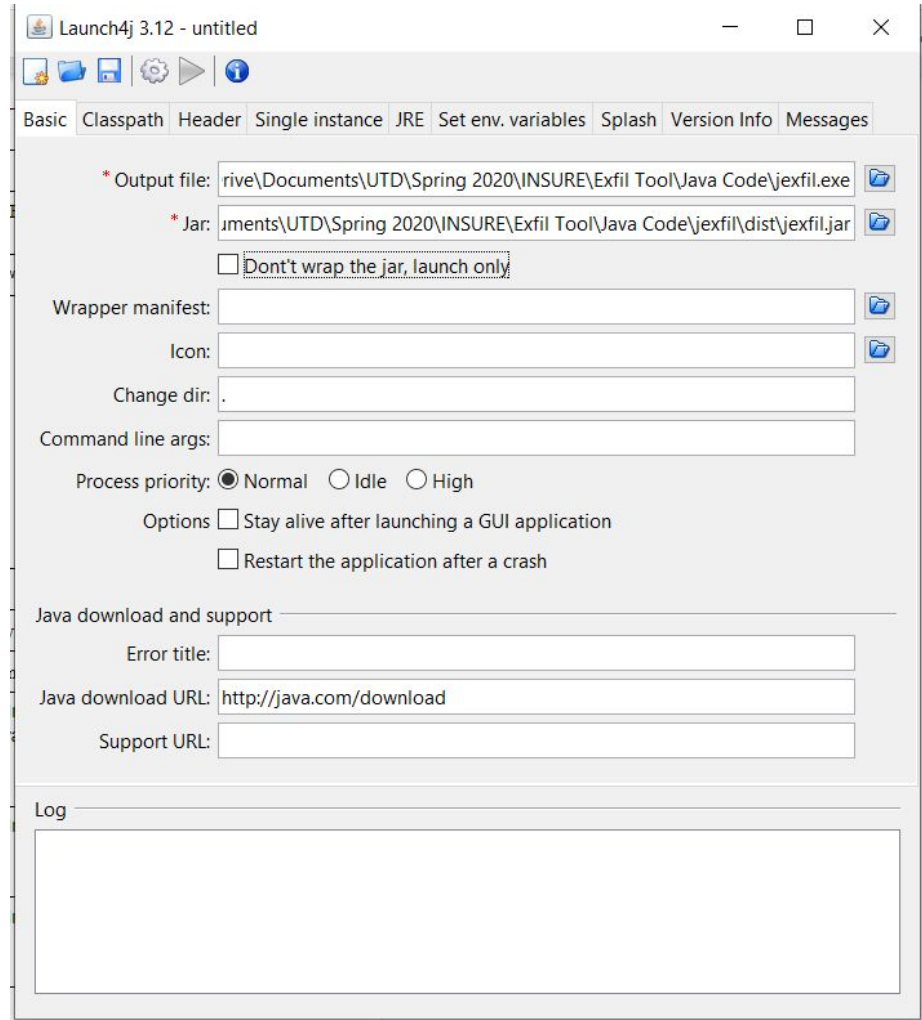
*setup.py - configuration for cx_freeze

```
from cx_Freeze import setup, Executable
setup(name = "exfilTool" ,
      version = "0.1" ,
      description = "" ,
      executables = [Executable("exfilTool.py")])
```

**The code was still written in Python 3, but PyInstaller ran from Python 2

Work Completed: Packaging

Language	Java
Tool	Launch4J
Options	N/A



Work Completed: Original Feature Extraction List

Features from Requirements:

- Type of tool (ex: data exfil)
- What does it do?

Features from Design:

- Input - config file, hard coded, command line, user input
- Use of functions (modular) vs no functions
- Data structures - arrays, linked lists, stacks, queues, trees, heap, hashing, etc.
- Algorithms used (obfuscation possibly)

Features from Code:

- *Language*
- Variable/function names
- How well does the author know the language (well written code vs poorly written)
- *Libraries used*

Features from Build:

- *Compiler*
- *Linker*
- *Options*
- *Obfuscation (compiling options)*

Features from Configure:

- Have we seen this tool before?
- *Which options are used?*
- How is it configured?

Features from Package:

- Where did we find it? Wrapper?
- Wrong file extension?
- Encoded in a different file?

Pefile

Timestamp for when executable was created

Architecture PE is supposed to be run on

Interpreted as a 32-bit or 64-bit binary

Warnings while parsing PE to determine anomalies (i.e. obfuscation)

Imported DLLs and their corresponding imported functions

Packers that match PEiD signatures

Peframe

Antidebug - Any DLLs or API function calls specific to anti-debugging

Antivm - Any DLLs or API function calls specific to anti-virtual

machines

Packers

Crypto

Files and file types in the strings

URLs in the strings

IP addresses in the strings

Fuzzing in the strings

Antivirus

Metadata:

Comments, Company Name, File Description, File Version,

Internal Name, Legal Copyright, Legal Trademarks,

Original Filename, Product Name, Product Version

Results

Searching strings

Programming language (Go, Java, Python)

Python-specific:

Padding (“PADDINGXX”) present

If exe created from pyinstaller, pyinstxtractor extracts:

Pyinstaller version

Python version

Length of package

Number of files in pyinstaller archive

Possible entry points

Java-specific:

Classes in source code package

Identified by ending in .classPK

Imported packages/classes/functions

Identified by java/*

Go-specific:

Function calls (from main):

Identified by main.*

Imported libraries/packages/functions:

Identified by vendor/golang.org/x/*

Identified by go.itab.*

Python-magic

File type

Analysis

- Snippet of output of PEFeatExt.py on a python exfil tool packaged into an executable using PyInstaller with standard options
- Displays extracted features from the previous slide
 - PyInstaller version 2.1+
 - Python 3.7 was used for the source code
 - File was created 01-05-2020

```
Filename: tool.exe
Programming Language: Python
Timestamp: 2020-01-05 12:15:27
PADDINGXX: False
Pyinstaller:
    Pyinstaller version: 2.1+
    Python version: 37
    Length of package: 7876903 bytes
    Found 82 files in CArchive
    Possible entry point: pyiboot01_bootstrap
    Possible entry point: pyi_rth_certifi
    Possible entry point: tool
File Type: PE32+ executable (console) x86-64, for MS Windows
Machine: 64-bit (x86-x64)
Magic: 64-bit binary
Metadata:
Crypto:
    CRC32_poly_Constant
    CRC32_table
Antidebug:
    GetLastError
    IsDebuggerPresent
    IsProcessorFeaturePresent
    RaiseException
    TerminateProcess
    UnhandledExceptionFilter
Antivm:
Packers:
    Microsoft_Visual_Cpp_80_DLL
Files:
    %s%s%s.exe                                Executable
    ucrtbase.dll                               Library
    base_library.zip                           Compressed
    KERNEL32.dll                               Library
    ADVAPI32.dll                               Library
    WS2_32.dll                                 Library
    'LI.gZ                                     Compressed
    0P      },Q/.sO                             Executable
    44.gZ                                       Compressed
    =p.msg                                     Email
    bVCRUNTIME140.dll                           Library
    bapi-ms-win-core-console-l1-1-0.dll         Library
    bapi-ms-win-core-datetime-l1-1-0.dll        Library
    . . . . .
```

Analysis

- We used “diff” to compare output of Standard, Optimized, and Encrypted PyInstaller versions of our exfiltration tools
- Standard and Optimized have almost identical output
- Encrypted detected a few files that were not in Standard or Optimized
 - Some of these files seem unlikely to be true files
 - “0P } , Q / . s O”
 - It is still good to search for these extensions
 - Malware authors may use obfuscation or encryption

Analysis

```
diff standardPyInstallerOutput optimizedPyInstallerOutput
```

```
8c8
```

```
<      Length of package: 7876903 bytes
```

```
---
```

```
>      Length of package: 7876941 bytes
```

Analysis

diff **standard**PyInstallerOutput **encrypted**PyInstallerOutput

```
8,9c8,9
<      Length of package: 7876903 bytes
<      Found 82 files in CArchive
---
>      Length of package: 7907538 bytes
>      Found 84 files in CArchive
39,40c39,41
<      44.gZ
<      =p.msg
---
>      qe-KW=.DB
>      g\ .Bmp
>      H`@3?+vX1} KQB^9RlRo.dB
```



Analysis

diff **encrypted**PythonOutput **optimized**PythonOutput

```
8,9c8,9
<      Length of package: 7907538 bytes
<      Found 84 files in CArchive
---
>      Length of package: 7876941 bytes
>      Found 82 files in CArchive
39,41c39,40
<      qe-KW=.DB                      Database
<      g\.Bmp                        Image
<      H`@3?+vX1j KQB^9RlRo.dB       Database
---
>      44.gZ                          Compressed
>      =p.msg                        Email
```

Challenges

- **Obfuscation:** It is difficult to detect common features among malware when the malware is obfuscated by its author or authors.
- **Author Variations:** An author or group of authors of malware may purposefully make variations in implementation language, configuration options, and other features when writing each new piece of malware to prevent this type of analysis from proving effective.
- **Features Not Detected in Final Product:** Some key “fingerprints” found in a certain phase of the MDLC may not be retrievable in the resulting executable that will be analyzed.
- **Finding Malware Samples and Ground Truth:** It’s difficult to find wild malware samples that you know a ground truth for - you don’t have the source code.

Future Work

- Additional feature extraction
- Finding additional similarities/differences in malware samples
- Finding features common to an author or group
- Testing with additional samples, especially wild malware
- Determine whether a given sample is malware or benign.

Questions?

References

- [1] Ranveer, Smita, and Swapnaja Hiray. "Comparative Analysis of Feature Extraction Methods of Malware Detection." *International Journal of Computer Applications*, vol. 120, no. 5, June 2015, pdfs.semanticscholar.org/8ec8/858a25ce2fb76388d40ad07878b0ec79f580.pdf.
- [2] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath. 2011. Malware images: visualization and automatic classification. In Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec '11). Association for Computing Machinery, New York, NY, USA, Article 4, 1–7. DOI:<https://doi.org/10.1145/2016904.2016908>
- [3] M. Ahmadi, D. Ulyanov, S. Semenov, M. Trofimov, G. Giacinto. March 2016. Novel Feature Extraction, Selection and Fusion for Effective Malware Family Classification. In CODASPY '16: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy. Association for Computing Machinery, New York, NY, USA, pages 183 - 194.
- [4] R. Tian, L.M. Batten, R. Islam, and S.C. Versteeg, "Differentiating Malware from Cleanware Using Behavioural Analysis," in Proc. of the 5th IEEE Int. Conf on Malicious and Unwanted Software (Malware'10), Nancy, France, pp. 23–30, October 2010
- [5] S. K. Pandey and B. M. Mehtre, "A Lifecycle Based Approach for Malware Analysis," 2014 Fourth International Conference on Communication Systems and Network Technologies, Bhopal, 2014, pp. 767-771.